

Lecture 6 • Wed 10/14/2026

CS160 Compilers — Fall 2026

Parsing I.

Yu Feng

Paper and pencil out – today's first question gets worked by hand.

01

Where we are.



Monday: characters → tokens, with a DFA — a machine with a fixed set of states and no way to remember how many of anything it has seen.

Today: tokens → tree. Every phase after today reads the tree and never looks at your text again.

Monday's machine can't count — and the text never says which tree.

A DFA has a fixed, finite memory. It can never check that parentheses balance: $(((. . .)))$ needs unbounded counting, and a fixed set of states cannot count. That is the *easy* half of why tokens aren't enough.

The hard half: the same token string can have two different trees, and nothing in the text says which one is meant. $1 - 2 - 3$ is 2 or -4 depending only on a choice nobody wrote down.

So the shape of the tree is not read off the program. It has to be *decided in advance* — and the thing that decides it is a **grammar**.

Which tree does this string mean — and who decides?

By the end you can take a grammar and a string and produce both the leftmost derivation and the parse tree, by hand, on paper.

You can say whether a grammar is *ambiguous* — whether one string can have two trees — and rewrite one so that precedence and associativity are forced by its shape.

You can trace a parser by hand and name which call consumed which token, for both of the parsers we write today — and **PA2 is that parser**.

A context-free grammar is a set of rewrite rules.

```
E -> E + T | E - T | T
T -> T * F | T // F | F
F -> INT | ( E )
```

```
nonterminals: E, T, F    (things that expand)
terminals:   + - * // INT ( )    (tokens)
start symbol: E
```

Read $E \rightarrow E + T$ as: an expression can be an expression, a plus, a term. The language is every token string reachable from E .

production

one rule: one nonterminal, one right-hand side. The first line is two productions sharing a left-hand side, written with $|$.

the punctuation

$+ - * // ()$ are written bare here because they *are* the tokens; slide 15 changes that convention.

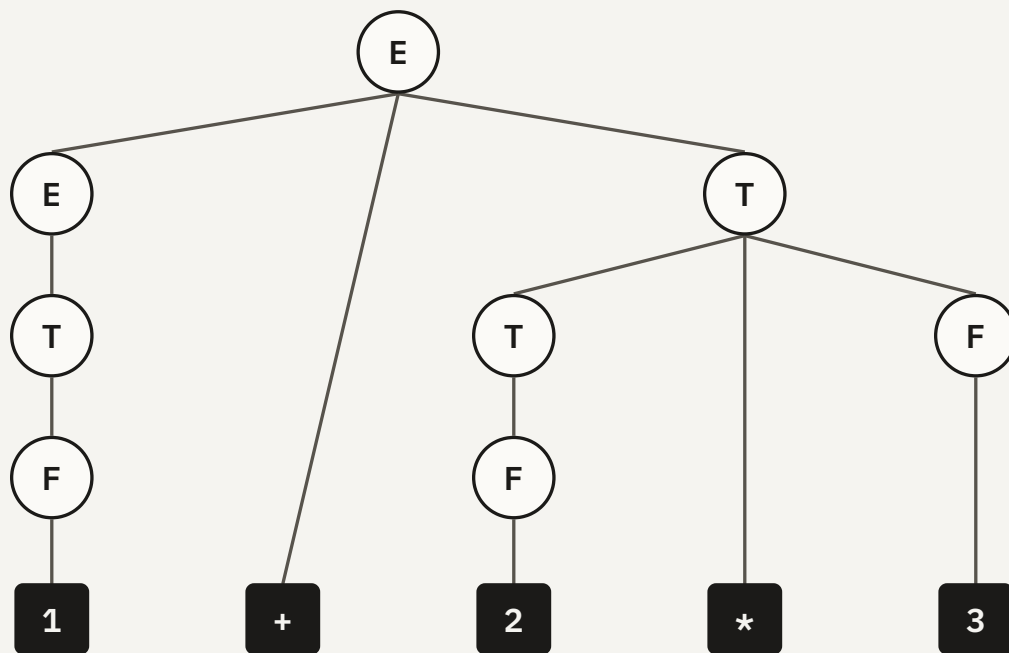
Rewrite the leftmost nonterminal until only tokens remain.

```
E => E + T => T + T => F + T => 1 + T => 1 + T * F
=> 1 + F * F => 1 + 2 * F => 1 + 2 * 3
```

derivation a sequence of rewrites. Read $\Rightarrow$ as “rewrites to” — one rule applied, and not the same arrow as the $\rightarrow$ inside a rule.

A *leftmost derivation* of $1 + 2 * 3$: each $\Rightarrow$ applies one rule to the leftmost nonterminal, so the only thing left to choose is *which* rule.

A parse tree is that derivation, drawn.



parse tree

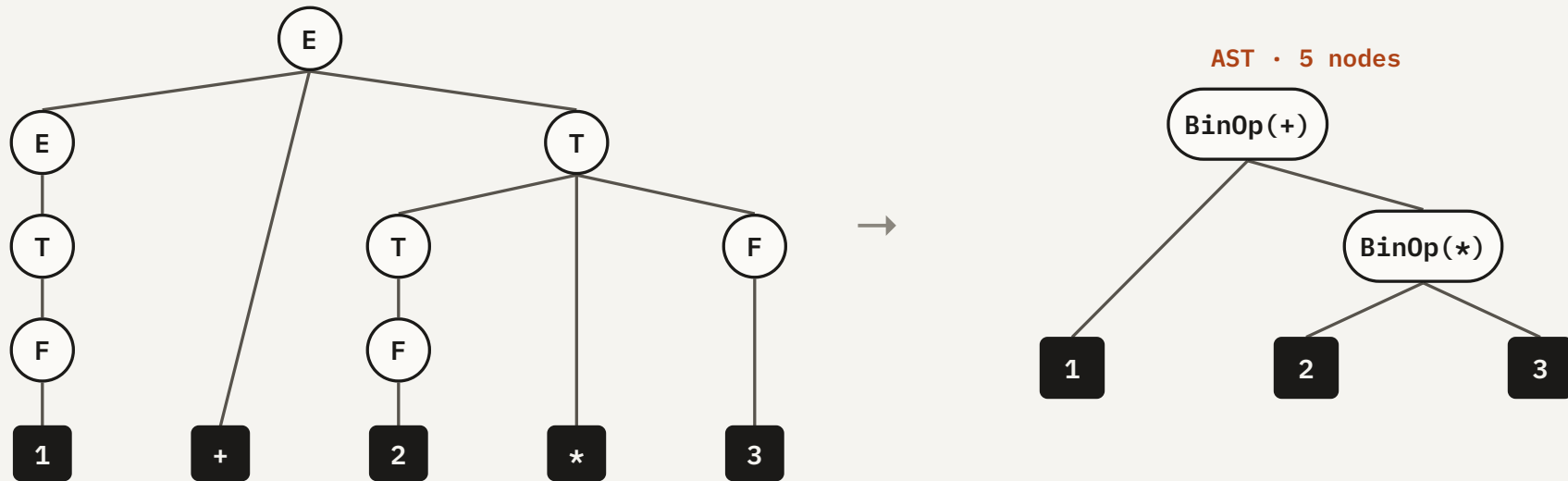
one node per rule application — the nonterminal, with its right-hand side as children. Leaves spell the input.

○ nonterminal

■ token

AST — *abstract syntax tree* — is the same shape with the grammar's bookkeeping thrown away.

```
parse tree · 13 nodes
```



The parse tree records *how* the grammar matched. The AST records *what was said*.

Chains like $E \rightarrow T \rightarrow F \rightarrow 1$ and the parentheses vanish.

Lecture 3's compiler consumes the AST and never sees a parse tree. Neither will yours.

Python's AST for the same thing.

```
$ python3 -c "import ast; print(ast.dump(ast.parse('1 + 2 * 3', mode='eval').body))"  
BinOp(left=Constant(value=1), op=Add(), right=BinOp(left=Constant(value=2), op=Mult(), right=Constant(value=3)))
```

The target. PA2's parser must build exactly these nodes with these field names: the oracle is string equality on `ast.dump`.

Terminals are token *kinds*.

```
F -> INT | ID | ( E )
```

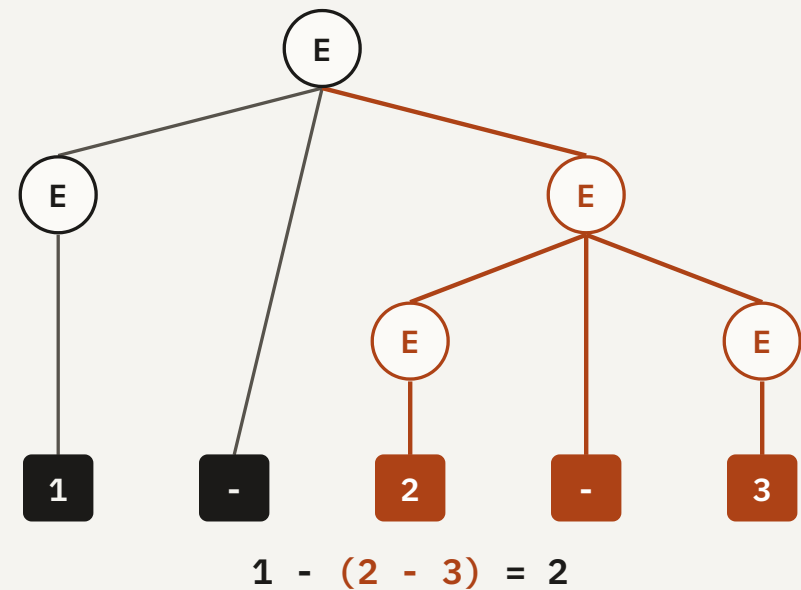
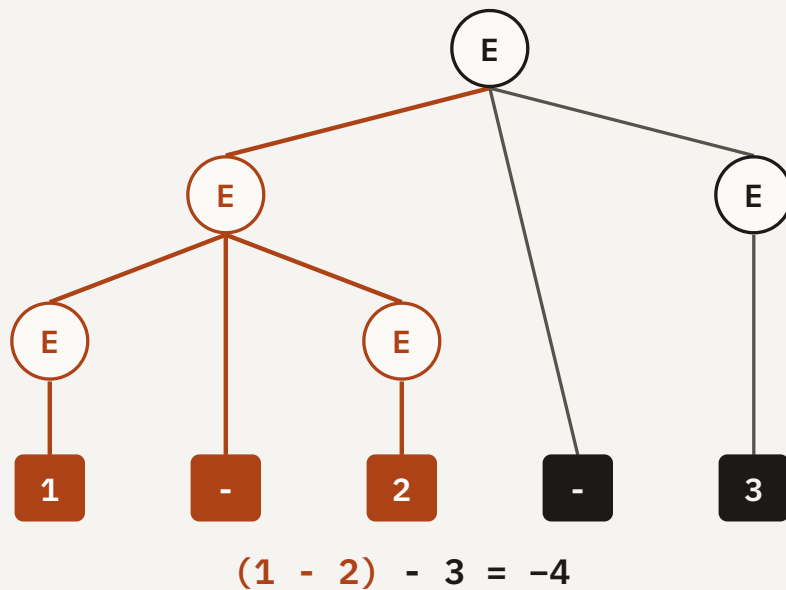
`INT`, `ID` are kinds — any integer, any name.

`(` is an OP token whose text is `(`.

The parser asks `tok.kind == "INT"`, never `tok.text.isdigit()`. The lexer already decided — that's the point of two phases.

Two trees for one string.

$E \rightarrow E - E \mid \text{INT}$



ambiguous Both are valid. A grammar with two parse trees for one string is *ambiguous*; it says nothing about which is right, so the parser picks one and you won't know which.

The dangling else.

```
if (a) if (b) x = 1; else x = 2;      /* C: which if owns the else? */
```

Ambiguous in C's grammar; the spec adds a rule: `else` binds to the nearest `if`.

ChocoPy has no dangling else: `INDENT/DEDENT` are the braces — the lexer resolved it Monday.

Ambiguity is a property of the grammar, not the language.

One nonterminal per precedence level.

precedence when two *different* operators compete for the same operand, which one gets it. $*$ binds tighter than $-$, so $1 - 2 * 3$ is $1 - (2 * 3)$.

$E \rightarrow E - T \mid T$

E : sums (lowest precedence)

$T \rightarrow T * F \mid F$

T : products

$F \rightarrow \text{INT} \mid (E)$

F : atoms (highest)

- lives only at level E; $*$ only at T. A $*$ can never be split by a $-$ beneath it. **Precedence, enforced by shape.**

Associativity is which side recurses.

associativity when the *same* operator repeats, which way it groups: $1 - 2 - 3$ as $(1 - 2) - 3$ (left-associative) or as $1 - (2 - 3)$ (right-associative). Precedence settles different operators; associativity settles equal ones.

$E \rightarrow E - T \mid T$

left-recursive: $(1 - 2) - 3$

✓ Python

$E \rightarrow T - E \mid T$

right-recursive: $1 - (2 - 3)$

✗ the puzzle

The side that recurses is the side that keeps growing. **Left-recursive rule, left-associative operator.** The puzzle recursed on the right.

The manual's grammar, and the table that disambiguates it.

ChocoPy v2.2 language manual, §4 — `::=` is `->`; `[ ... ]` groups, `?` is “optional”, `*` is “repeat”; terminals are bare.

```
expr ::= cexpr
      | not expr
      | expr [and | or] expr
      | expr if expr else expr
```

```
cexpr ::= ID
      | literal
      | [ [expr [, expr]*]? ]
      | ( expr )
      | member_expr
      | index_expr
      | member_expr ( [expr [, expr]*]? )
      | ID ( [expr [, expr]*]? )
      | cexpr bin_op cexpr
      | - cexpr
```

```
bin_op ::= + | - | * | // | % | == | != | <= | >= | < | > | is
```

```
index_expr ::= cexpr [ expr ]
```

```
member_expr ::= cexpr . ID
```

```
literal ::= None | True | False | INTEGER | STRING
```

Two nonterminals, not eight — and `cexpr bin_op cexpr` is **ambiguous on its own**: it is slide 11's `E -> E - E` with a bigger operator table, so it gives `1 - 2 - 3` both trees. Nothing in these productions decides.

What decides is a table printed *beside* the grammar.

ChocoPy v2.2 manual, §4.1, verbatim — nine levels, loosest first.

Precedence	Operator(s)	Associativity
1	<code>· if · else ·</code>	Right
2	<code>or</code>	Left
3	<code>and</code>	Left
4	<code>not</code>	N/A
5	<code>==, !=, <, >, <=, >=, is</code>	None
6	<code>+, - (binary)</code>	Left
7	<code>*, //, %</code>	Left
8	<code>- (unary)</code>	N/A
9	<code>., []</code>	Left

“Note that the comparison operators are nonassociative (so ChocoPy, unlike Python 3, does not allow expressions such as $x < y < z$).”

§4.1

Level 5 says **None**. That is the $a < b < c$ ban, written down by the people who wrote the language — and it is why our comparison rule takes `?` and not `*`.

The same table, encoded into the grammar.

parser.py · docstring — one nonterminal per level, low → high

```
or_expr    -> and_expr ('or' and_expr)*
and_expr   -> not_expr ('and' not_expr)*
not_expr    -> 'not' not_expr | cmp_expr
cmp_expr    -> arith ( ('==' | '!=' | '<' | '<=' | '>' | '>=') arith )?
arith       -> term ( ('+' | '-') term )*
term        -> unary ( ('*' | '/' | '%') unary )*
unary       -> '-' unary | atom
atom        -> INT | 'True' | 'False' | ID | '(' or_expr ')'
```

the notation just changed from here on **terminals are quoted** ('*' is the multiplication token, 'or' the keyword), and the *unquoted* () * ? are notation: (...) groups, * is “zero or more of the group before it”, ? is “zero or one”. On the `term` line the quoted '*' multiplies and the bare * at the end repeats. Slide 5 was the other way round.

Eight levels, written with * (repeat) instead of left recursion — same trees, and the form **recursive descent** (one function per nonterminal — next slide) wants. `cmp_expr` uses ?: one comparison, no chains.

Two places the table can live.

In the grammar (ours)

Stratifying *is* the table, moved into the grammar: one nonterminal per level, so the tree's shape carries the precedence and nothing is looked up while parsing. The standard move for a hand-written parser — and what you build in PA2.

Beside the grammar (generators)

A parser generator does not need it: hand it the manual's ambiguous `cexpr bin_op cexpr` **plus** the §4.1 table as precedence declarations, and it resolves the conflicts for you. Monday.

Not implemented, for now: classes, `for`, conditional expressions (`· if · else ·`, level 1), `member_expr (·)` and `is`. Nine manual levels minus level 1 is our eight; `atom` sits where level 9 does, without `·` or `[]`, and with no lists, strings, `None` or calls yet.

One function per nonterminal. The call stack is the parse tree.

recursive descent begin at the start symbol and *descend* — one function per nonterminal, each calling the functions for the nonterminals on its right-hand side, so the recursion has the grammar's shape.

parser.py

```
def not_expr(self):
    if self.at("KW", "not"):
        self.expect("KW", "not")
        return ast.UnaryOp(op=ast.Not(), operand=self.not_expr())
    return self.cmp_expr()

def unary(self):
    if self.at("OP", "-"):
        self.expect("OP", "-")
        return ast.UnaryOp(op=ast.USub(), operand=self.unary())
    return self.atom()
```

Look at the next token; pick the alternative; consume, recurse, build the node. peek, at, expect are the only helpers.

The helpers.

```
def peek(self):    return self.toks[self.i]
def at(self, kind, text=None):
    t = self.peek(); return t.kind == kind and (text is None or t.text == text)
def expect(self, kind, text=None):
    if not self.at(kind, text):
        t = self.peek(); raise ParseError(f"line {t.line}, col {t.col}: expected {text or kind!r} ...")
    self.i += 1; return self.toks[self.i - 1]
```

expect is where every syntax error is born. It has the token, so it has line and column.

Monday's tokens paid off.

Atoms and parentheses.

```
def atom(self):
    t = self.peek()
    if t.kind == "INT": self.i += 1; return ast.Constant(value=int(t.text))
    if t.kind == "ID": self.i += 1; return ast.Name(id=t.text, ctx=ast.Load())
    if self.at("OP", "("):
        self.expect("OP", "("); e = self.or_expr(); self.expect("OP", ")"); return e
```

Parentheses: recurse to the *top* level and come back. No node is built — the tree already has the right shape.

Why the grammar says $*$ and not $E \rightarrow E - T$

written literally from $E \rightarrow E - T$

```
def arith(self): left = self.arith(); ...
```

It calls itself before consuming anything: **RecursionError**.

A rule that starts with its own nonterminal can't be a function that starts by calling itself. The $(' - ' \text{ term})^*$ loop is the fix. Monday: the theory, and the mechanical rewrite.

The puzzle, and ours.

```
$ python3 puzzle.py
```

```
BinOp(left=Constant(value=1), op=Sub(), right=BinOp(left=Constant(value=2), op=Sub(), right=Constant(value=3)))  
evaluates to 2  
Python says -4
```

```
$ python3 parser.py "1 - 2 - 3"
```

```
OK 1 - 2 - 3  
BinOp(left=BinOp(left=Constant(value=1), op=Sub(), right=Constant(value=2)), op=Sub(), right=Constant(value=3))
```

OK means `ast.dump(ours) == ast.dump(ast.parse(src))`. Lecture 4's oracle, one phase earlier.

Both arithmetic levels in one function.

precedence climbing one function instead of one per level, carrying as an argument the lowest precedence it is still allowed to accept — it *climbs* that floor as it descends into an operand.

```
parser.py · PREC = {"+": 1, "-": 1, "*": 2, "//": 2, "%": 2}
```

```
def arith(self, min_prec):
    left = self.unary()
    while self.at("OP") and self.peek().text in PREC and PREC[self.peek().text] >= min_prec:
        op = self.expect("OP").text
        right = self.arith(PREC[op] + 1)      # +1: left-associative
        left = ast.BinOp(left=left, op=BINOP[op](), right=right)
    return left
```

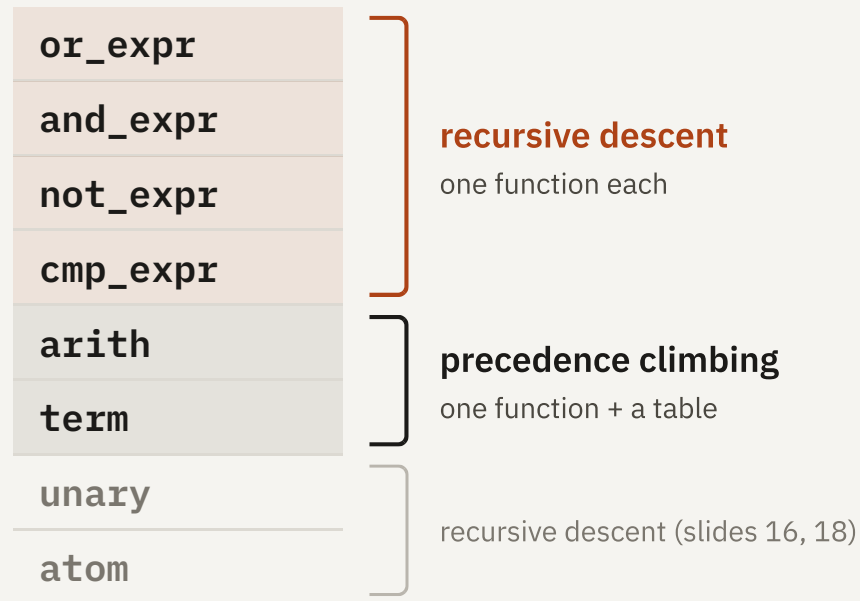
Parse a unary. While the next operator binds at least as tightly as allowed: consume it, parse the right side allowing only *tighter* operators, combine. One table, not one function per level.

Trace: 1 + 2 * 3

```
arith(0): left=1; see + (1>=0): right = arith(2)
    arith(2): left=2; see * (2>=2): right = arith(3) = 3; left = 2*3
    left = 1 + (2*3)
```

`min_prec` is the whole trick: inside the right operand of `+`, a `*` may continue but another `+` may not — it returns to the outer loop.

Both kinds in one parser.



Mixing is normal. not and comparisons have quirks (prefix keyword, no chaining) a table handles awkwardly; arithmetic is pure table.

The oracle on twelve expressions.

```
$ python3 parser.py "1 - 2 - 3" "1 + 2 * 3" ... "(a or b) or c"
```

OK 1 - 2 - 3

OK 1 + 2 * 3

OK (1 + 2) * 3

OK -x * y (-x) * y

OK - -x

OK a < b + 1

OK not a == b not (a == b)

OK a and b or c

OK a or b or c one BoolOp, 3 values

OK x % 2 == 0 and y

OK 2 * 3 // 4 % 5

OK (a or b) or c nested BoolOp

Twelve OKs, including the ones that bite: `-x * y` is `(-x) * y`; `not a == b` is `not (a == b)`; `a or b or c` is *one* BoolOp with three values — Python flattens, so we flatten.

Live · about a minute

Lecture 3's compiler, unchanged.

expr2ll.py · lex → our parser → compile_stmt from cocoa3.py → clang

```
$ python3 expr2ll.py "(1 + 2) * 3 - 4 // 2"
7
```

the .ll it wrote

```
%t1 = add i32 1, 2
%t2 = mul i32 %t1, 3
%t3 = sdiv i32 4, 2
%t4 = sub i32 %t2, %t3
call void @print_int(i32 %t4)
```

cocoa3.py is byte-for-byte last Monday's file. Build the same nodes and every later phase is free. **That's PA2's contract.**

Monday.

Statements and the indentation grammar; PA2 due; PA3 out.

The theory: which grammars parse with one token of lookahead (LL(1)) and how to know; bottom-up parsing; parser generators.

Midterm 1 is a week from today; Monday ends with the review.